

Lab 4 — Vibe Coding Chat History

Project: 08_helicopter — Python/Pygame Helicopter Game

Purpose: Record the Vibe Coding workflow used to fix the starter game and implement Tasks 1–4.

Assignment Workflow

The lab required an individual workflow: read the README, clone/fork the assigned repository, run the original game, record a before video, use Vibe Coding prompts to fix the broken code and add the README features, record an after video, push the code, and keep separate commits for each task.

Repository Setup

- Opened the assigned repository: SETAPESU26/08_helicopter.
- Read the README.
- Forked the repository to the student's GitHub account.
- Cloned the fork locally and opened it in VS Code.
- Found the actual project inside the nested helicopter folder.
- Installed pygame-ce because Python 3.14.2 could not build the repository's Pygame dependency from source.
- Ran python main.py successfully.
- Recorded the required before video before modifying the code.

Vibe Coding — Task 1: Movement and Boundaries

User prompt:

I am working on a Python Pygame helicopter game for my Vibe Coding lab.

Please fix ONLY the helicopter movement and boundary issues.

Requirements:

1. The helicopter should respond quickly when the player switches between UP and DOWN.
2. Prevent the vertical velocity from becoming excessively large.
3. Prevent the helicopter from going above the top of the screen.
4. Prevent the helicopter from going below the bottom of the screen.
5. Use the existing height_bound parameter in update().
6. Keep the existing class structure and get_rect() function.
7. Do not implement collision detection, distance scoring, shield, or any other features yet.
8. Make the minimum necessary changes.

Follow-up prompt after reviewing the first response:

Review your previous solution again against the original helicopter.py.

The original get_rect() treats x and y as the CENTER of the helicopter. Preserve the original get_rect() exactly.

The requirement says UP/DOWN switching should be responsive. Simply limiting accumulated velocity may still cause a delay. Re

- UP sets a fast upward velocity.
- DOWN sets a fast downward velocity.
- No key stops vertical movement.
- Vertical speed is capped.
- Top and bottom boundaries use the helicopter's center and half-height.
- Do not implement collision, scoring, shield, or other features.

Final Task 1 code applied to game/helicopter.py:

```
import pygame

THRUST = 0.4
MAX_VERTICAL_SPEED = 6.0

class Helicopter:
    def __init__(self, x, y, width=40, height=24):
        self.x = x
```

```

        self.y = y
        self.width = width
        self.height = height
        self.vy = 0.0

    def handle_input(self, keys_pressed):
        if keys_pressed[pygame.K_UP]:
            self.vy = -MAX_VERTICAL_SPEED
        elif keys_pressed[pygame.K_DOWN]:
            self.vy = MAX_VERTICAL_SPEED
        else:
            self.vy = 0.0

    def update(self, height_bound):
        self.y += self.vy

        if self.y - self.height / 2 < 0:
            self.y = self.height / 2
            self.vy = 0.0

        if self.y + self.height / 2 > height_bound:
            self.y = height_bound - self.height / 2
            self.vy = 0.0

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2),
            int(self.y - self.height / 2),
            self.width,
            self.height,
        )

```

Testing result: movement, immediate direction switching, and top/bottom boundaries were tested successfully.

Commit: Fix helicopter movement and boundaries

Vibe Coding — Task 2: Collision and Game Over

Relevant existing files inspected: game_engine.py, obstacle.py, helicopter.py.

Task 2 prompt:

Now implement Task 2 of my helicopter game assignment.

Requirements:

1. Detect collision between the helicopter and obstacles.
2. Use the existing helicopter.get_rect().
3. Use obstacle.get_top_rect() and obstacle.get_bottom_rect().
4. Collision with either wall must cause Game Over.
5. Flying through the gap must not cause Game Over.
6. Once Game Over occurs, normal gameplay/movement should stop.
7. Display a clear "GAME OVER" message.
8. Preserve Task 1 movement and boundary fixes.
9. Do not implement distance scoring or shield yet.
10. Make the minimum necessary changes.

Task 2 implementation:

```

self.game_over = False

def handle_input(self, keys_pressed):
    if self.game_over:
        return
    self.helicopter.handle_input(keys_pressed)

def update(self):
    if self.game_over:
        return

    self.helicopter.update(HEIGHT)

    # spawn/update obstacles ...

    for obstacle in self.obstacles:
        obstacle.update()

        if self.helicopter.get_rect().colliderect(
            obstacle.get_top_rect()
        ):
            self.game_over = True

```

```

        return

    if self.helicopter.get_rect().colliderect(
        obstacle.get_bottom_rect()
    ):
        self.game_over = True
        return

```

The draw method was also updated to display a clear GAME OVER message. Testing confirmed top-wall collision, bottom-wall collision, and safe passage through the gap.

Commit: Add collision detection and game over

Vibe Coding — Task 3: Distance Score

Task 3 implementation:

```

self.distance = 0

def update(self):
    if self.game_over:
        return

    self.helicopter.update(HEIGHT)

    # Increase distance while playing
    self.distance += SCROLL_SPEED

    # existing obstacle update and collision logic ...

def draw(self, surface, font):
    # existing scene drawing ...

    distance_text = font.render(
        f"Distance: {self.distance}",
        True,
        (255, 255, 255)
    )
    surface.blit(distance_text, (10, 10))

    # After Game Over, show final distance as well.

```

Testing confirmed the distance increased during gameplay, appeared on screen, remained visible after Game Over, and started at zero for a new game.

Commit: Add distance scoring

Vibe Coding — Task 4: Shield Mechanic

Task 4 implementation:

```

self.shield_active = False

def handle_keydown(self, key):
    if self.game_over:
        return

    if key == pygame.K_s:
        self.shield_active = True

# During collision checking:
collision = (
    helicopter_rect.colliderect(top_rect)
    or helicopter_rect.colliderect(bottom_rect)
)

if collision:
    if self.shield_active:
        self.shield_active = False
    else:
        self.game_over = True
        return

# In draw():
if self.shield_active:
    shield_text = font.render(
        "SHIELD ACTIVE",
        True,

```

```

        (0, 255, 255)
    )
    surface.blit(shield_text, (10, 40))

    pygame.draw.circle(
        surface,
        (0, 255, 255),
        (int(self.helicopter.x), int(self.helicopter.y)),
        28,
        3
    )

```

Shield key: S. Testing confirmed that the shield activates visibly, absorbs one obstacle collision, disappears after absorbing the hit, allows the game to continue, and a later collision causes Game Over.

Commit: Add shield mechanic

Final Git Commit History

- Add shield mechanic
- Add distance scoring
- Add collision detection and game over
- Fix helicopter movement and boundaries
- Initial Helicopter starter

Final Verification

- ✓ Before video recorded before code changes.
- ✓ After video recorded after all four tasks.
- ✓ Four separate task commits created.
- ✓ Changes pushed to the student's fork.
- ✓ Updated code prepared for the Lab-4 submission folder.

Note: This document summarizes the actual Vibe Coding workflow carried out in this ChatGPT conversation, including the prompts, implementation decisions, testing, and commit history.